Trạng thái	Đã xong
Bắt đầu vào lúc	Thứ Bảy, 10 tháng 8 2024, 10:12 AM
Kết thúc lúc	Thứ Sáu, 23 tháng 8 2024, 3:55 PM
Thời gian thực hiện	13 Các ngày 5 giờ
Điểm	4,00/4,00
Điểm	**10,00** trên 10,00 (**100**%)

Câu hỏi 1

Đúng

Đạt điểm 1,00 trên 1,00

Implement function

```
int binarySearch(int arr[], int left, int right, int x)
```

to search for value x in array arr using recursion.
After traverse an index in array, we print out this index using cout << "We traverse on index: " << index << endl;

Note that middle of left and right is floor((right-left)/2)

For example:

Test	Result

int arr[] = {1,2,3,4,5,6,7,8,9,10};
int x = 10;
int n = sizeof(arr) / sizeof(arr[0]);
int result = binarySearch(arr, 0, n - 1, x);
(result == -1) ? cout << "Element is not present in array"
: cout << "Element is present at index " << result;
	We traverse on index: 4
	We traverse on index: 7
	We traverse on index: 8
	We traverse on index: 9
	Element is present at index 9

Answer: (penalty regime: 0, 0, 5, ... %)

Reset answer

```
int binarySearch(int arr[], int left, int right, int x) {
    if (right >= left) {
        int mid = left + (right - left) / 2;

        // Print the current index being traversed
        cout << "We traverse on index: " << mid << endl;

        // Check if x is present at mid
        if (arr[mid] == x)
            return mid;

        // If x is smaller, it can only be in the left subarray
        if (arr[mid] > x)
            return binarySearch(arr, left, mid - 1, x);

        // Else, x can only be in the right subarray
        return binarySearch(arr, mid + 1, right, x);
    }

    // If x is not present in the array
    return -1;
}
```

	Test	Expected	Got	

✓	int arr[] = {1,2,3,4,5,6,7,8,9,10};
	int x = 10;
	int n = sizeof(arr) / sizeof(arr[0]);
	int result = binarySearch(arr, 0, n - 1, x);
	(result == -1) ? cout << "Element is not present in array"
	: cout << "Element is present at index " << result;
		We traverse on index: 4
		We traverse on index: 7
		We traverse on index: 8
		We traverse on index: 9
		Element is present at index 9
			We traverse on index: 4
			We traverse on index: 7
			We traverse on index: 8
			We traverse on index: 9
			Element is present at index 9
				✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.

Câu hỏi **2**

Đúng

Đạt điểm 1,00 trên 1,00

Implement function

```
int interpolationSearch(int arr[], int left, int right, int x)
```

to search for value x in array arr using recursion.
After traverse to an index in array, before returning the index or passing it as argument to recursive function, we print out this index using cout << "We traverse on index: " << index << endl;

Please note that you can't using key work for, while, goto (even in variable names, comment).

For example:

Test	Result

`int arr[] = { 1,2,3,4,5,6,7,8,9 };`
`int n = sizeof(arr) / sizeof(arr[0]);`
`int x = 3;`
`int result = interpolationSearch(arr, 0, n - 1, x);`
`(result == -1) ? cout << "Element is not present in array"`
`: cout << "Element is present at index " << result;`
	`We traverse on index: 2`
	`Element is present at index 2`

`int arr[] = { 1,2,3,4,5,6,7,8,9 };`
`int n = sizeof(arr) / sizeof(arr[0]);`
`int x = 0;`
`int result = interpolationSearch(arr, 0, n - 1, x);`
`(result == -1) ? cout << "Element is not present in array"`
`: cout << "Element is present at index " << result;`
	`Element is not present in array`

Answer: (penalty regime: 0, 0, 5, ... %)

Reset answer

```
int interpolationSearch(int arr[], int left, int right, int x)
{
    if (right>=left && x>=arr[left] && x<=arr[right]) {
        int pos= left+((x-arr[left])*(right-left)/(arr[right]-arr[left]));
        if (pos<0) return -1;
        cout << "We traverse on index: " << pos << endl;
        if (arr[pos]==x) return pos;
        else if (x<arr[pos]) return interpolationSearch(arr, left, pos-1, x);
        else return interpolationSearch(arr, pos+1, right, x);
    }
    return -1;
}
```

	Test	Expected	Got	

✓	int arr[] = { 1,2,3,4,5,6,7,8,9 };
	int n = sizeof(arr) / sizeof(arr[0]);
	int x = 3;
	int result = interpolationSearch(arr, 0, n - 1, x);
	(result == -1) ? cout << "Element is not present in array"
	: cout << "Element is present at index " << result;
		We traverse on index: 2
		Element is present at index 2
			We traverse on index: 2
			Element is present at index 2
				✓

✓	int arr[] = { 1,2,3,4,5,6,7,8,9 };
	int n = sizeof(arr) / sizeof(arr[0]);
	int x = 0;
	int result = interpolationSearch(arr, 0, n - 1, x);
	(result == -1) ? cout << "Element is not present in array"
	: cout << "Element is present at index " << result;
		Element is not present in array	Element is not present in array	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.

Câu hỏi **3**

Đúng

Đạt điểm 1,00 trên 1,00

In computer science, a jump search or block search refers to a search algorithm for ordered lists. The basic idea is to check fewer elements (than linear search) by jumping ahead by fixed steps or skipping some elements in place of searching all elements. For example, suppose we have an array arr[] of size n and block (to be jumped) size m. Then we search at the indexes arr[0], arr[m], arr[2m].....arr[km] and so on. Once we find the interval (arr[km] < x < arr[(k+1)m]), we perform a linear search operation from the index km to find the element x. The optimal value of m is √n, where n is the length of the list.

In this question, we need to implement function jumpSearch with step √n to search for value x in array arr. After searching at an index, we should print that index until we find the index of value x in array or until we determine that the value is not in the array.

```
int jumpSearch(int arr[], int x, int n)
```

For example:

Test	Result

`int arr[] = { 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610 };`
`int x = 55;`
`int n = sizeof(arr) / sizeof(arr[0]);`
`int index = jumpSearch(arr, x, n);`
 	
`if (index != -1) {`
`  cout << "\nNumber " << x << " is at index " << index;`
`}`
`else {`
`  cout << "\n" << x << " is not in array!";`
`}`
	`0 4 8 12 9 10`
	`Number 55 is at index 10`

`int arr[] = { 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610 };`
`int x = 144;`
`int n = sizeof(arr) / sizeof(arr[0]);`
`int index = jumpSearch(arr, x, n);`
 	
`if (index != -1) {`
`  cout << "\nNumber " << x << " is at index " << index;`
`}`
`else {`
`  cout << "\n" << x << " is not in array!";`
`}`
	`0 4 8 12`
	`Number 144 is at index 12`

Answer: (penalty regime: 0 %)

Reset answer

```
int jumpSearch(int arr[], int x, int n) {
    // TODO: print the traversed indexes and return the index of value x in array if x is found, otherwise, r
    int left=0;
    int step=sqrt(n);
    while (left*step<n) {
        cout<<left*step<<" ";
        if(arr[left*step] >= x) break;
        left++;
        if (left*step>=n-1) break;
    }
    if (arr[left*step] == x) return left*step;
    for (int j=(left-1)*step+1; j<left*step; j++) {
        cout<<j<<" ";
        if(arr[j] == x) return j;
```

```
    }
    return -1;
}
```

	Test	Expected	Got	

✓	int arr[] = { 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610 };
	int x = 55;
	int n = sizeof(arr) / sizeof(arr[0]);
	int index = jumpSearch(arr, x, n);
	
	if (index != -1) {
	cout << "\nNumber " << x << " is at index " << index;
	}
	else {
	cout << "\n" << x << " is not in array!";
	}
		0 4 8 12 9 10
		Number 55 is at index 10
			0 4 8 12 9 10
			Number 55 is at index 10
				✓

✓	int arr[] = { 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610 };
	int x = 144;
	int n = sizeof(arr) / sizeof(arr[0]);
	int index = jumpSearch(arr, x, n);
	
	if (index != -1) {
	cout << "\nNumber " << x << " is at index " << index;
	}
	else {
	cout << "\n" << x << " is not in array!";
	}
		0 4 8 12
		Number 144 is at index 12
			0 4 8 12
			Number 144 is at index 12
				✓

✓	int arr[] = { 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 611, 612, 613 };
	int x = 612;
	int n = sizeof(arr) / sizeof(arr[0]);
	int index = jumpSearch(arr, x, n);
	
	if (index != -1) {
	cout << "\nNumber " << x << " is at index " << index;
	}
	else {
	cout << "\n" << x << " is not in array!";
	}
		0 4 8 12 16 17
		Number 612 is at index 17
			0 4 8 12 16 17
			Number 612 is at index 17
				✓

	Test	Expected	Got	

✓	`int arr[] = { 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 611, 612, 613 };`
	`int x = 614;`
	`int n = sizeof(arr) / sizeof(arr[0]);`
	`int index = jumpSearch(arr, x, n);`
	
	`if (index != -1) {`
	`  cout << "\nNumber " << x << " is at index " << index;`
	`}`
	`else {`
	`  cout << "\n" << x << " is not in array!";`
	`}`
		`0 4 8 12 16 17 18 19`
		`614 is not in array!`
			`0 4 8 12 16 17 18 19`
			`614 is not in array!`
				✓

✓	`int arr[] = { 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 611, 612, 613, 1000, 1002, 2000, 2003, 2004, 2005, 2006 };`
	`int x = 36;`
	`int n = sizeof(arr) / sizeof(arr[0]);`
	`int index = jumpSearch(arr, x, n);`
	
	`if (index != -1) {`
	`  cout << "\nNumber " << x << " is at index " << index;`
	`}`
	`else {`
	`  cout << "\n" << x << " is not in array!";`
	`}`
		`0 5 10 6 7 8 9`
		`36 is not in array!`
			`0 5 10 6 7 8 9`
			`36 is not in array!`
				✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.

Câu hỏi **4**

Đúng

Đạt điểm 1,00 trên 1,00

Given an array of distinct integers, find if there are two pairs (a, b) and (c, d) such that a+b = c+d, and a, b, c and d are distinct elements. If there are multiple answers, you can find any of them.

Some libraries you can use in this question:

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <algorithm>
#include <iostream>
#include <utility>
#include <map>
#include <vector>
#include <set>
```

Note: The function checkAnswer is used to determine whether your pairs found is true or not in case there are two pairs satistify the condition. You don't need to do anything about this function.

For example:

Test	Result

`int arr[] = { 3, 4, 7, 1, 2, 9, 8 };`
`int n = sizeof arr / sizeof arr[0];`
`pair<int, int> pair1, pair2;`
`if (findPairs(arr, n, pair1, pair2)) {`
`  if (checkAnswer(arr, n, pair1, pair2)) {`
`  printf("Your answer is correct.\n");`
`  }`
`  else printf("Your answer is incorrect.\n");`
`}`
`else printf("No pair found.\n");`
	`Your answer is correct.`

`int arr[] = { 3, 4, 7 };`
`int n = sizeof arr / sizeof arr[0];`
`pair<int, int> pair1, pair2;`
`if (findPairs(arr, n, pair1, pair2)) {`
`  if (checkAnswer(arr, n, pair1, pair2)) {`
`  printf("Your answer is correct.\n");`
`  }`
`  else printf("Your answer is incorrect.\n");`
`}`
`else printf("No pair found.\n");`
	`No pair found.`

Answer: (penalty regime: 0 %)

Reset answer

```
bool findPairs(int arr[], int n, pair<int,int>& pair1, pair<int, int>& pair2) {
    // Create a map to store sums of pairs
    map<int, pair<int, int>> sumMap;

    // Traverse through all pairs
```

```
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            int sum = arr[i] + arr[j];

            // If the sum is already in the map, we found two pairs with the same sum
            if (sumMap.find(sum) != sumMap.end()) {
                pair<int, int> prevPair = sumMap[sum];

                // Ensure that all elements are distinct
                if (prevPair.first != arr[i] && prevPair.first != arr[j] &&
                    prevPair.second != arr[i] && prevPair.second != arr[j]) {

                    // Assign the pairs to the output parameters
                    pair1 = {prevPair.first, prevPair.second};
                    pair2 = {arr[i], arr[j]};

                    return true;
                }
            } else {
                // Store the sum in the map
                sumMap[sum] = {arr[i], arr[j]};
            }
        }
    }

    // If no such pairs are found, return false
    return false;
}
```

	Test	Expected	Got	

✓	`int arr[] = { 3, 4, 7, 1, 2, 9, 8 };`
	`int n = sizeof arr / sizeof arr[0];`
	`pair<int, int> pair1, pair2;`
	`if (findPairs(arr, n, pair1, pair2)) {`
	`  if (checkAnswer(arr, n, pair1, pair2)) {`
	`  printf("Your answer is correct.\n");`
	`  }`
	`  else printf("Your answer is incorrect.\n");`
	`}`
	`else printf("No pair found.\n");`
		Your answer is correct.	Your answer is correct.	✓

✓	`int arr[] = { 3, 4, 7 };`
	`int n = sizeof arr / sizeof arr[0];`
	`pair<int, int> pair1, pair2;`
	`if (findPairs(arr, n, pair1, pair2)) {`
	`  if (checkAnswer(arr, n, pair1, pair2)) {`
	`  printf("Your answer is correct.\n");`
	`  }`
	`  else printf("Your answer is incorrect.\n");`
	`}`
	`else printf("No pair found.\n");`
		No pair found.	No pair found.	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.